

Manipulations avancées de données



Bienvenue à notre module sur la **manipulation avancée des données**. Ces techniques sont essentielles pour une programmation efficace. Ce module aborde:

- Fonction `map()`, pour appliquer une fonction à chaque élément d'un iterable.
- Fonction `filter()`, pour créer une nouvelle liste basée sur une condition.
- Fonction `reduce()`, qui combinées ensemble, peuvent simplifier la réduction des listes.
- Fonction `lambda`, des fonctions anonymes pour les tâches très courtes.
- Fonction récursives, où une fonction se réfère elle-même pour résoudre des problèmes complexes.



La compréhension de liste



Fonctions `map()`, `filter()` et `reduce()`



Les fonctions `lambda`

 **Les fonctions récursives**

 **Review**

La compréhension de liste

Définition

La **compréhension de liste** (*list comprehension* en anglais) est une fonctionnalité du langage Python qui va nous permettre de créer des listes de manière concise et facile à lire.

Grâce aux compréhensions de listes, il est possible de filtrer, **transformer ou combiner des éléments d'une liste existante** pour en créer une nouvelle en une seule et unique instruction.

Syntaxe et décodage de la compréhension de liste

La syntaxe d'une *list comprehension* répond au modèle suivant :

```
nouvelleList = [expression for listItem in mylist conditions]
```

Syntaxe d'une list comprehension.

Ce modèle de code nécessite un minimum d'explications.

expression

L'expression est la modification que nous souhaitons appliquer à chaque élément de la liste de départ. C'est cette expression qui va déterminer le contenu de la nouvelle liste qui sera créée à l'issue de l'opération.

listItem

Il s'agit d'une variable qui représentera successivement chacun des éléments de la liste d'origine. Cette variable peut être nommée comme bon vous semble.

myList

myList est la liste d'origine à partir de laquelle la nouvelle liste sera créée.

conditions

Ce paramètre optionnel dans la structure est une expression conditionnelle destinée à filtrer les éléments de la liste d'origine. Seuls les éléments qui satisfont la condition (évaluée à True) seront traités pour générer la nouvelle liste.

Quelques exemples

Exemple 1 : supposons que nous avons une liste de nombres et que nous voulons transformer cette liste de sorte que chaque élément soit mis au carré pour obtenir une nouvelle liste.

```
listeStart = [2,4,6,8,10]
listeEnd = []

for i in range(len(listeStart)):
    listeEnd.append(listeStart[i]**2)

print(listeStart)
print(listeEnd)
```

Script qui crée une seconde liste à partir d'éléments (élevés au carré) d'une liste.

Après initialisation de la liste de destination, une boucle parcourt tous les éléments de la liste de départ, calcule le carré de chaque élément et range le résultat dans la seconde liste.

Le résultat produit par ce code sera le suivant :

```
[2, 4, 6, 8, 10]
[4, 16, 36, 64, 100]
```

Résultat du script précédent.

Mais il est possible d'**utiliser *list comprehension* pour obtenir le même résultat** :

```
listeStart = [2,4,6,8,10]
listeEnd =[elem**2 for elem in listeStart]

print(listeStart)
print(listeEnd)
```

Script qui utilise la list comprehension pour obtenir le même résultat que précédemment.

Exemple 2 : soit une liste de nombres. L'objectif est de créer une seconde liste comprenant uniquement les éléments compris entre 50 et 100 exprimés en pourcentage.

De manière « classique », nous pouvons procéder de la sorte :

```
listeStart = [20,40,60,70,80,100]
listeEnd = []

for i in range(len(listeStart)):
    if(50 <= listeStart[i] <= 100):
        listeEnd.append(f"{listeStart[i]}%")

print(listeStart)
print(listeEnd)
```

Script qui crée une seconde liste ne comprenant que les éléments compris entre 50 et 100 d'une première liste. Dans la seconde liste, chaque élément est exprimé en pourcentage.

Après initialisation de la liste de destination, une boucle parcourt tous les éléments de la liste de départ, vérifie si l'élément est compris entre 50 et 100 et, le cas échéant, le range dans la liste de destination. Finalement, le contenu des deux listes est affiché.

Le résultat produit par ce code sera le suivant :

```
[20, 40, 60, 70, 80, 100]
```

```
['60%', '70%', '80%', '100%']
```

Nous pouvons simplement **utiliser une *list comprehension*** assortie d'une condition pour obtenir le même résultat :

```
listeStart = [20,40,60,70,80,100]
listeEnd =[f"{elem}%" for elem in listeStart if 50 <= elem <= 100]

print(listeStart)
print(listeEnd)
```

Script qui utilise list comprehension pour effectuer le même traitement.

CONTINUER

Fonctions `map()`, `filter()` et `reduce()`

Il existe quelques fonctions très pratiques qui permettent d'optimiser les traitements.

Il s'agit de fonctions facilitant le **traitement des séquences**. Elles permettent de manipuler des données de manière fonctionnelle.

Nous allons en voir 3 intégrées au langage :

- La fonction `map()`
- La fonction `filter()`
- La fonction `reduce()`

La fonction `map()`

La fonction `map()` permet d'appliquer une fonction à chaque élément d'une séquence (liste, tuple, set). Le résultat retourné est une nouvelle séquence contenant les résultats.

La fonction `map()` a la syntaxe suivante :

```
resultat = map(fonction, sequence)
```

La syntaxe de la fonction `map()`.

Imaginons que nous souhaitons doubler la valeur de chaque élément d'une liste :

```
def traitement(x):  
    return x*2  
  
sequence = [1,2,3,4,5]  
resultat = list(map(traitement, sequence))  
  
print(resultat)
```

Script qui applique une fonction à chaque élément d'une liste et utilise chaque résultat pour créer une seconde liste.

Dans cet exemple :

- Nous **définissons la fonction à utiliser dans la fonction `map()`**: `traitement(x)`. Cette fonction a pour objectif de doubler la valeur de son paramètre.
- Nous **définissons la séquence** que nous allons traiter.
- Nous **utilisons la fonction `map()` en lui passant la fonction à appliquer ainsi que la séquence à traiter** et nous **convertissons le résultat** en une liste au moyen de la **fonction `list()`**.

Ce qui donne le résultat suivant :

```
[2, 4, 6, 8, 10]
```

Résultat du script précédent.

La fonction `filter()`

La fonction `filter()` permet, comme son nom l'indique, de **filtrer les éléments d'une séquence** en y appliquant une fonction qui évalue une ou un ensemble de conditions.

Si la fonction appliquée à l'élément courant de la séquence renvoie `True`, l'élément sera intégré au résultat, si par contre le résultat est `False`, l'élément sera exclu.

Le fonctionnement de la fonction `filter()` répond au modèle suivant :

```
resultat = filter(fonction, sequence)
```

Syntaxe de la fonction `filter()`.

Dans ce modèle, **fonction** est la fonction de filtrage à appliquer à la **séquence** passée en paramètre.

Par exemple, filtrons les nombres impaires d'une séquence sans utilisation de la fonction `filter()` :

```
myList = [1,2,3,4,5,6,7,8,9,10]
myResult = []

for i in range(len(myList)):
    if(myList[i]%2==0):
        myResult.append(myList[i])

print(myResult)
```

Script qui crée une liste des éléments pairs d'une liste originelle.

Pour chaque élément de notre liste, vérifions la parité de l'élément courant et si cette parité est rencontrée, mettons l'élément courant dans la liste de résultats.

Utilisons la fonction `filter()` :

```
def test(x): return x % 2 == 0

myList = [1,2,3,4,5,6,7,8,9,10]
myResult = list(filter(test, myList))

print(myResult)
```

Script qui crée une liste d'éléments pairs présents dans une liste originelle.

Dans cet exemple :

- Nous **définissons la fonction** à utiliser dans la fonction `filter()`: la fonction `test(x)`. Cette fonction a pour objet de doubler la valeur du paramètre qui lui est transmis.
- Nous **définissons la séquence** que nous allons traiter.

- Nous **utilisons la fonction filter()** en lui communiquant la fonction à appliquer ainsi que la séquence à traiter. Ensuite, nous **convertissons le résultat** en une liste au moyen de la **fonction list()**.

Ce qui nous produit le résultat suivant :

```
[2, 4, 6, 8, 10]
```

Résultat du script précédent.

La fonction reduce()

La fonction reduce(), bien que native, nécessite l'importation du module **functools**.

Ainsi, pour pouvoir utiliser la fonction, le code commencera avant toute chose par son importation au moyen de l'instruction import :

```
from functools import reduce
```

Commande effectuant l'importation de la fonction reduce depuis le module functools.

Le fonctionnement de reduce() répond au modèle suivant :

```
resultatUnique = filter(fonction, sequence)
```

Syntaxe de la fonction `filter()`.

Dans ce modèle, **fonction** est la fonction d'agrégation à appliquer sur **sequence**, la séquence passée en paramètre.

`reduce()`, comme `filter()` et `map()`, va permettre d'appliquer une fonction à une séquence mais, cette fois, à des fins d'agrégation de cette séquence. En d'autres mots, après application de la fonction `reduce()`, **nous obtiendrons une valeur unique**.

La fonction `reduce()` est, comme son nom l'indique, une fonction qui effectue une réduction des éléments d'une séquence.

Cette réduction est effectuée en appliquant une fonction cumulative de gauche à droite soit du premier au dernier élément de la séquence.

Voyez plutôt l'exemple suivant dans lequel on calcule la somme de tous les éléments d'une liste :

```
from functools import reduce

def traitement(x, y):
    return x+y

sequence = [1,2,3,4,5]
resultat = reduce(traitement, sequence)

print(resultat)
```

Script qui calcule la somme de tous les éléments d'une liste.

Nous avons :

1. importé la fonction `reduce()` ;
2. défini la fonction de réduction *traitement* qui aura pour objectif d'additionner l'élément de gauche de notre séquence avec son élément suivant ;
3. défini la séquence que nous allons utiliser ;
4. appliqué la fonction `reduce()` en utilisant notre fonction *traitement* et notre séquence.

Le résultat de ce traitement sera :

15

15 étant la somme des éléments de notre liste, le résultat est dès lors correct.

 On peut déduire de cet exemple que la fonction `sum()` native applicable aux listes est donc une réduction de cette liste pour ainsi dire telle que nous l'avons défini dans l'exemple.

CONTINUER

Les fonctions lambda

Vous avez probablement remarqué dans la leçon précédente qu'il était obligatoire, lors de l'utilisation des fonctions `map()`, `filter()` et `reduce()`, de définir la fonction à appliquer sur notre séquence.

L'**avantage** ? Cette fonction peut être réutilisée dans le code.

L'**inconvénient** ? Devoir définir une fonction pour un traitement extrêmement court et probablement à usage unique.

La **fonction lambda** est la solution à ce problème et permet d'utiliser des fonctions `map()`, `filter()` ou autres de manière encore plus concise.

Une **fonction lambda** ou **fonction anonyme** est une fonction **sans nom** qui peut être définie de manière *inline* (en une seule ligne).

Contrairement aux fonctions définies au moyen du mot clé `def` qui doivent être nommées, les fonctions lambda ont pour vocation d'être utilisées pour des opérations simples qui se font en une seule expression.

Pour définir une fonction lambda, nous utiliserons la syntaxe suivante :

```
lambda paramètres: traitement
```

Syntaxe d'utilisation de la fonction lambda.

Dans cette syntaxe, **lambda** est le mot clé réservé à la déclaration des fonctions lambda, **paramètres** représente le ou les paramètres passés à la fonction et séparés par une virgule et **traitement** est l'expression unique que la fonction va effectuer et pour laquelle un résultat sera retourné.

Si nous reprenons l'exemple de la leçon précédente où nous souhaitons multiplier par 2 toutes les entrées de notre séquence au moyen de la fonction `map()` :

```
def traitement(x):  
    return x*2  
  
sequence = [1,2,3,4,5]  
resultat = list(map(traitement, sequence))  
  
print(resultat)
```

Le script sans utilisation de la fonction lambda.

Et si nous mettons en œuvre une expression lambda, nous obtenons le code suivant :

```
sequence = [1,2,3,4,5]  
resultat = list(map(lambda x: x*2, sequence))  
  
print(resultat)
```

Le script avec utilisation de la fonction lambda.

Ce code est **plus léger, plus concis et plus facile à lire**. Le traitement quant à lui reste inchangé.

Exemple d'utilisation de la fonction `reduce()`:

```
from functools import reduce

def traitement(x, y):
    return x+y

sequence = [1,2,3,4,5]
resultat = reduce(traitement, sequence)

print(resultat)
```

Script utilisant la fonction `reduce()` sans fonction lambda.

Même chose que pour l'exemple précédent, faisons lui subir un petit lifting :

```
from functools import reduce

sequence = [1,2,3,4,5]
resultat = reduce(lambda x,y: x+y, sequence)

print(resultat)
```

Script utilisant la fonction `reduce()` avec une fonction lambda.

Le résultat retourné par une fonction lambda peut également être **stockée dans une variable**. Il suffit pour cela d'affecter à une variable l'expression lambda.

Prenons l'exemple d'une fonction ayant pour but de déterminer si un nombre est pair :

```
est_pair = lambda x: x%2==0

print(est_pair(2))
print(est_pair(3))
```

Script qui vérifie la parité d'un nombre au moyen d'une fonction lambda.

Cette fonction est callable à souhait dans le code.

Le script renverra le résultat suivant :

```
True
False
```

Résultat du script précédent.

Voilà donc là toute l'utilité des lambdas :

- Rendre le code plus clair.
- Rendre le code plus concis.
- Augmenter la lisibilité du code.

CONTINUER

Les fonctions récursives

Définition

Une fonction récursive est une fonction qui s'appelle elle-même.

Avec les fonctions récursives, il y a un élément capital auquel il faut prêter attention : **prévoir une condition d'arrêt.**

Cette condition d'arrêt permettra d'empêcher que la fonction tourne de manière récursive **indéfiniment** entraînant par la même occasion une erreur de consommation mémoire dans le code et donc un crash de l'application.

Une fonction récursive peut très bien être écrite en remplacement d'une structure itérative.

Fonctionnement d'une fonction récursive

Une fonction récursive est une fonction qui cherche à satisfaire une condition qui arrêtera son exécution. Tant que cette condition d'arrêt n'est pas satisfaite, on procèdera à un éventuel ajustement des paramètres de la fonction avant que celle-ci se rappelle elle-même pour (re)tenter de valider la condition d'arrêt.

Exemple 1 - Afficher un décompte

Nous allons, sur base d'une valeur introduite par l'utilisateur, afficher un décompte allant de ce nombre à 0.

Le premier traitement sera fait de manière classique :

```
start = int(input("Entrez un nombre : "))

while start>0 :
    print(start)
    start = start - 1
else:
    print("Go!")
```

Script qui réalise un traitement répétitif sans usage de la récursivité.

Ce qui produira le résultat suivant :

```
Entrez un nombre : 5
5
4
3
2
1
Go!
```

Le résultat du script précédent.

Si maintenant nous utilisons une fonction récursive pour faire la même chose :

```
def decomppte(start):
    if start>0:
        print(start)
        decomppte(start-1)
    else:
        print("Go!")

start = int(input("Entrez un nombre : "))
decomppte(start)
```

Script qui remplit le même objectif que précédemment mais avec l'usage de la récursivité.

Exemple 2 - Trouver l'indice d'un élément dans une collection

Soit un script permettant de découvrir la position d'un élément dans une séquence.

Sans fonction récursive :

```
maSequence = ["jean","claudio", "eudes", "david", "julia", "katarina"]
valueToFind = "julia"
result = None

for i, value in enumerate(maSequence):
    if value == valueToFind:
        result = i
        break;

if(result==None):
    print(f"{valueToFind} n'est pas présent dans la séquence")
else:
    print(f"{valueToFind} se trouve dans la séquence en position {result}")
```

Script qui recherche la position d'un élément sans faire usage de la récursivité.

Au moyen d'une fonction récursive :

```
def indexOf(value, array, index=0):
    if(index==len(array)): return None

    if(array[index]==value): return index
    else:
        index = index + 1
        return indexOf(value, array, index)

maSequence = ["jean","claudio", "eudes", "david", "julia", "katarina"]
valueToFind = "julia"

result = indexOf(valueToFind, maSequence)

if(result==None):
    print(f"{valueToFind} n'est pas présent dans la séquence")
else:
    print(f"{valueToFind} se trouve dans la séquence en position {result}")
```

Script qui recherche la position d'un élément en faisant usage de la récursivité.

Que remarquons nous :

- Nous avons défini une fonction pour faire le traitement.
- Nous n'avons plus de boucle dans notre code.
- Ce code, dû à son écriture modulaire, peut être réutilisé.

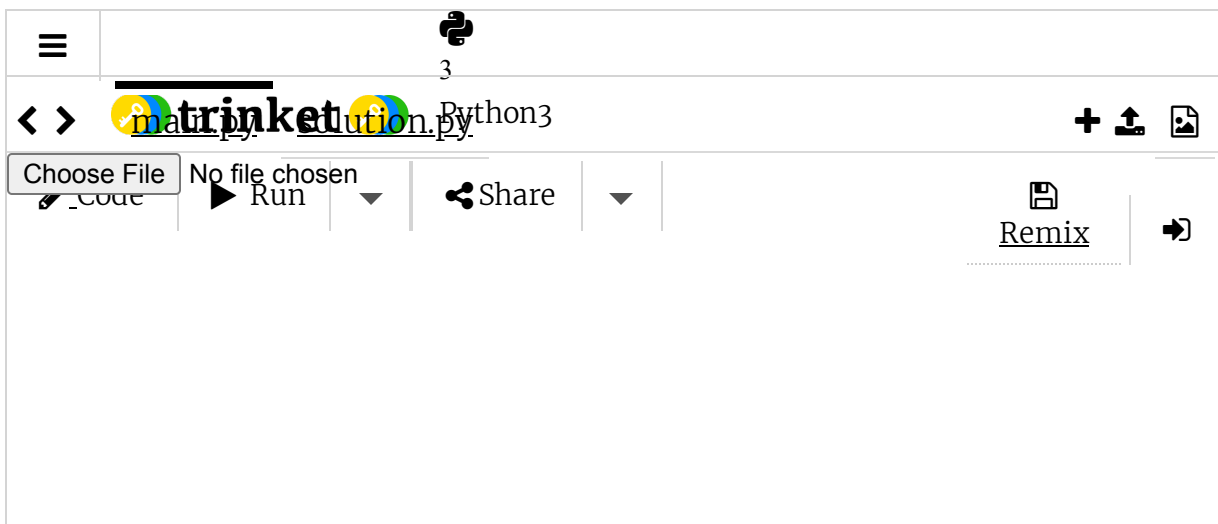
- La ligne de traitement est unique dans le cas de l'appel de la fonction récursive.

CONTINUER

Review

Soit la collection suivante : `[1,2,3,4,5,6,7,8,9,10]`.

Utilisez la fonction `map()` afin d'obtenir comme résultat cette même collection dont seulement les nombres pairs auront été élevés au carré.



CONTINUER

Soit la collection suivante : $[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$.

Au moyen d'une expression lambda et de la fonction appropriée, récupérez les nombres impairs de cette collection.



CONTINUER

Mettez en place une fonction récursive permettant de calculer $n!$ ($n!$ étant la notation mathématique de la factorielle du nombre n).

Pour rappel, la factorielle d'un nombre est le produit (la multiplication) de tous les nombres entiers inférieurs ou égaux à ce nombre sauf 0 (et en toute logique 1).

Ainsi, la factorielle de 3, notée $3!$ vaut $3 \times 2 \times 1 = 6$ et la factorielle de 5, notée $5!$ vaut $5 \times 4 \times 3 \times 2 \times 1 = 120$.

Attention, il n'est possible de calculer une factorielle que pour des nombres entiers positifs. Tenez en compte dans votre script et gérez l'erreur au besoin.



CONTINUER